

Operating Systems - Chapter 28 Summary

Locks: Synchronization and Mutual Exclusion

Locks are synchronization mechanisms that ensure mutual exclusion — allowing only one thread at a time to execute a critical section. They prevent race conditions, which occur when threads concurrently modify shared data without coordination.

Motivating Example: Race Condition

```
int counter = 0;

void *worker(void *arg) {
    for (int i = 0; i < 1000000; i++)
        counter++; // race condition
    return NULL;
}
```

Multiple threads increment the same counter, leading to inconsistent results. Locks ensure atomic access to shared variables, fixing this issue.

Naive Software Lock

```
typedef struct __lock_t {
    int flag;
} lock_t;

void init(lock_t *lock) { lock->flag = 0; }

void lock(lock_t *lock) {
    while (lock->flag == 1); // busy-wait
    lock->flag = 1;
}

void unlock(lock_t *lock) { lock->flag = 0; }
```

This fails because multiple threads can pass the check before setting the flag, violating mutual exclusion. We need **atomic hardware instructions** to guarantee correctness.

Test-And-Set (TAS)

```
int TestAndSet(int *ptr, int new) {
    int old = *ptr;
    *ptr = new;
    return old;
}

void lock(lock_t *lock) {
    while (TestAndSet(&lock->flag, 1) == 1);
}

void unlock(lock_t *lock) { lock->flag = 0; }
```

Test-and-set is atomic — it reads and sets memory in one instruction. Only one thread will acquire the lock, but spinning wastes CPU cycles.

Compare-And-Swap (CAS)

```
int CompareAndSwap(int *ptr, int expected, int new) {
    int old = *ptr;
    if (old == expected)
```

```

        *ptr = new;
        return old;
    }

    void lock(lock_t *lock) {
        while (CompareAndSwap(&lock->flag, 0, 1) != 0);
    }
    void unlock(lock_t *lock) { lock->flag = 0; }

```

CAS atomically changes a value only if it matches the expected one. This supports non-blocking algorithms and lock-free synchronization.

Fairness: Ticket Lock

```

typedef struct __ticketlock_t {
    int ticket;
    int turn;
} ticketlock_t;

void lock(ticketlock_t *lock) {
    int myturn = FetchAndAdd(&lock->ticket, 1);
    while (lock->turn != myturn);
}
void unlock(ticketlock_t *lock) {
    lock->turn++;
}

```

Ticket locks ensure fairness: threads acquire the lock in order of arrival. However, all threads still spin on the same variable, causing high contention on multicore CPUs.

Sleeping Locks (Futex-based)

```

if (TestAndSet(&lock->flag, 1) == 1)
    futex_wait(&lock->flag, 1); // put thread to sleep
else
    ; // acquired lock

void unlock(lock_t *lock) {
    lock->flag = 0;
    futex_wake(&lock->flag, 1); // wake one waiting thread
}

```

Futex (fast userspace mutex) combines spinning and kernel-assisted sleeping. Threads spin briefly, then sleep, avoiding CPU waste under contention.

Deadlocks

A deadlock occurs when two or more threads hold resources the others need, and none can proceed. Four conditions (Coffman, 1971): mutual exclusion, hold-and-wait, no preemption, circular wait. To avoid deadlocks, enforce lock ordering or use trylock-based avoidance.

Evaluating Locks

Good locks balance correctness, fairness, and performance. Spinning locks perform well for short critical sections, while sleeping locks are better for long waits.

Metrics: • Correctness – mutual exclusion and freedom from deadlock. • Fairness – each thread eventually acquires the lock. • Performance – low latency and minimal context switching. • Scalability – handles many cores efficiently.

Summary

Locks are fundamental synchronization primitives ensuring mutual exclusion. Hardware support (TAS, CAS, FAA) enables atomicity. Fairness and efficiency can be achieved using ticket locks and futexes. Designers must balance spinning vs sleeping, and prevent deadlocks by enforcing disciplined lock usage.